

CSCE 451/851 Programming Assignment 0

In this programming assignment you will be introduced to

- C/C++
- Linux/Unix POSIX library
- Manual pages or `man` pages
- Makefiles
- Development options and how to upload your code to the School of Computing servers.
- Understanding file streams

You will then develop two programs that emulate a few basic shell functions.

Submission procedure

Do the following:

1. Create a directory called `<UNL username>_pa0` (replace `<UNL username>` with your UNL username).
2. Place your code for `newecho` and `newcat` in the directory (They can be `.c` or `.cpp` files)
3. Add a `Makefile` with targets of
 - "newecho" -- should compile your `newecho` program into a binary called "newecho"
 - "newcat" -- should compile your `newcat` program into a binary called "newcat"
 - "all" -- should compile both programs
4. Zip the directory `<UNL username>_pa0` and submit (only three files are included, i.e., `newecho.cpp`, `newcat.cpp`, `Makefile`).

Use Canvas to submit your assignment. Submit a single zip file, `<UNL username>_pa0.zip` (e.g., `hguo_pa0.zip`).

Preliminary Information and References

C/C++ Programming

- There are lots of resources online to learn C. C and C++ are very similar, though C++ is a superset of C.
- You can use either C or C++ for any of the programming assignments. However, for most of them there is a choice that does make it a bit easier.
 - Most students use C++.

Resources for C

- [Beej's guide](#). Beej's guide also has lots of good information Unix interprocess communication (which you'll need)
- <https://www.tutorialspoint.com/cprogramming/index.htm>
- <https://www.w3schools.in/c-tutorial/>
- [C for Java Programmers](#)

Resources for C++

- [cplusplus](#) -- good tutorial and reference
- [learncpp](#) -- good tutorial
- [Very extensive reference](#)
- [Learning C++ when you know Python](#)

Main Differences Between C/C++

- Please review usage of pointers and references in C/C++, dynamic memory allocation (`malloc` and `free`)
- The primary difference between C and C++ is the use of objects in C++.
 - C has no concept of "objects" and therefore everything is procedural.
 - The only slight advantage C++ may have in this course is it's native handling of strings.

Linux/Unix and POSIX Library

- There are two main methods for accessing Linux/Unix OS functions: System V and POSIX
- System V was developed in 1983 at AT&T and many descendants of that system are still based on it.
- In 1988 the IEEE Computer Society decided to standardize this creating the Portable Operating System Interface (POSIX) standard for maintaining compatibility between Unix variants.
- Most operating systems are now either POSIX compliant, or *mostly* POSIX compliant.
 - This includes Mac, Windows, Android, VxWorks, iOS, and most distributions of Linux.
 - In this class I strongly suggest you use the POSIX API as it will make your code interoperable between OSs. I also think many of the POSIX functions are easier to use. **You will need to pay attention to this when you look at "man" pages to make sure you're not accidentally looking at a System V page.** The function should indicate whether or not it is POSIX compliant.

POSIX C Library

[Here is a list of POSIX C library functions](#). These are also available as man pages in your shell.

man (manual) Pages

- `man` (or manual) pages are key to your programming assignments.
- `man` pages are simply manual pages created to help users understand how commands are used in Linux.
- These pages are installed, by default, alongside the commands. As long as the software developer created a man page, it'll be there to help explain how to use a specific command.
- `man` pages can be accessed within a *nix shell via the command `man <topic>`.
- Many topics relating the Unix kernel, C programming, and even some abstract topics are available. A pretty good list exists [here](#) or [here](#).

Makefile

- `make` is a tool to build a software project. It's similar to Ant for Java (if you're familiar with it).
- It uses a file named `Makefile` or `makefile` to do this.
- You will use `make` in this class to build your software projects.
 - You will need to write your own `Makefile` for some programming assignments, while for others it will be given.
 - Either way you should have an idea of how to write a `Makefile`, how to use it, and do basic debugging on it.

Some good resources are here:

- [GNU Makefile Overview](#)
- <http://www.cs.colby.edu/maxwell/courses/tutorials/maketutor/>
- <https://opensource.com/article/18/8/what-how-makefile>
- Or just google "Makefile tutorial" and you will get lots of great hits.

Suggestions (Optional)

If you plan to use your own machine for development, one way to make your life significantly easier is to setup Makefile targets that can automatically:

1. upload your code to the School of Computing servers
2. compile your code
3. run your code against test cases

This allows you to build that check into your development process.

Development Workflow and Uploading to School of Computing Servers

You can choose how you want to develop your programs. Generally, the two main options are:

1. Develop locally on your own machine, then check correctness by uploading your project to the School of Computing servers and recompiling and running tests there.
2. Develop and test directly on the School of Computing machines/servers. If you do this, please be very careful using VSCode. VSCode liberally uses its cache space and it quickly fills up your disk space allocation and you will get a "Disk Quota Exceeded" message from the sysadmins.

Most students choose the first option. In that case, there are numerous ways to get your code onto the School of Computing servers and compile and execute your code. Check out the [UNL School of Computing FAQ](#) for options.

Unix File Streams

- In the man pages, some of them use the term "stream," to discuss I/O while others use "descriptor." [Read this for an explanation.](#)
- One of the core philosophies in Unix, is that everything is a file (or treated like a file, to be more specific).
 - Files, directories, devices (e.g., terminal, storage, processor) are all treated as files.
 - This means we can open, read, and write to the devices, console, directory etc. the same way as a file.
 - Given this abstraction, it is often easier to visualize the data as just a stream of bytes. [This article talks more about it.](#)

Whenever Unix creates a process, it opens three files (or streams) by default.

1. `stdin` (file descriptor value = `STDIN_FILENO` = 0)
2. `stdout` (file descriptor value = `STDOUT_FILENO` = 1)
3. `stderr` (file descriptor value = `STDERR_FILENO` = 2)

Unix File Streams (cont.)

- These streams are handles to default input and output.
- `stdout` and `stderr` are usually mapped to the terminal (or console). This is the reason any print statement appears on the terminal.
- One of the nice parts about treating things as "streams" is that we don't need to worry about where the data is going.
 - So if, for example, we decide that we don't want the output to appear on the terminal, we can remap the output descriptor to a file instead.
 - Similarly we could read from a file instead of terminal by remapping `stdin` to something else.
- From the perspective of the process, it just needs to know the descriptor it should read and write to.
- The pipe command `|`, and the file redirector commands `>`, `<` work in this way.

Program Specifications (Assignment)

- In these exercises we will develop simplified versions of some of the Unix tools (`cat`, `echo`) and use the program to introduce the POSIX API and some basic Unix concepts.
- For these, we are looking for simple implementations.
 - You don't need to implement any switches or complex functionality (see Evaluation section below for more detail).

Program 1: `newecho`

- In this exercise write a C/C++ program that will echo back to the terminal the input given to the program.
- You can learn more about `echo` by typing `man echo` into a terminal. This exercise will help you learn to work with command line arguments. Remember the function prototype for `main` in C/C++ is

```
int main(int argc, char* argv[]);
```

or

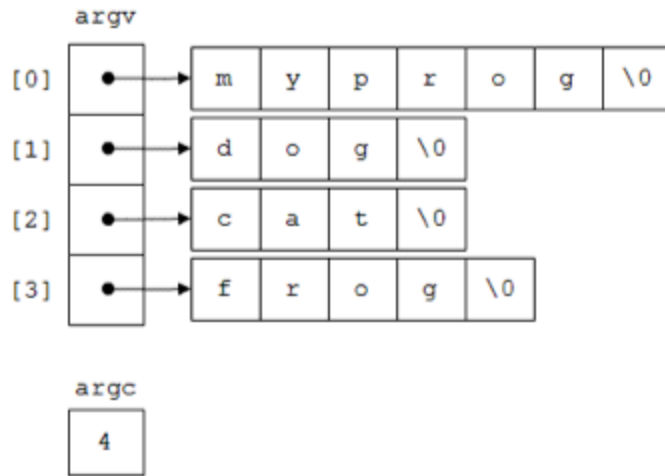
```
int main(int argc, char** argv);
```

where `argc` is the number of arguments and `argv` is a pointer to an array of pointers (remember C has no notion of strings).

Program 1: newecho (cont.)

- Each member of the array points to the characters in each argument to the program. A representation of this can be seen here

```
z123456@turing:~$ myprog dog cat frog
```



You can read more about it by Googling "argc and argv in C." At this point you just need to parse the arguments and print them back to the terminal. A `for` loop should do the job nicely.

Program 2: `newcat`

- Here you will write a C/C++ program that will concatenate and print the contents of the files presented as command line arguments.
- Read the man page on `cat` to learn more. This will help familiarize you with opening, closing, and reading files.

The steps to do this exercise are:

1. read in and parse the arguments
2. for each argument (let's assume no bad arguments)
 - a. open the file
 - b. read the contents of the file
 - c. print contents of file to terminal
 - d. close file

Program 2: `newcat` (cont.)

There are roughly two main strategies for doing the file I/O:

1. Use the C functions in `stdio.h`. These are C functions that utilize the underlying system libraries. As such they're perhaps a little cleaner and easier to use.
2. Use the underlying system POSIX API. These are very similar to the C library though perhaps a bit more "raw."

Typically, the C library functions look like `fopen`, `fclose`, etc. and work with "streams" represented by `FILE`. The POSIX system functions look like `open`, `close`, etc. and work with a file descriptor (which is really just an `int`). These types are nearly identical and can be converted to the other easily.

Feel free to use either one you want, but it is generally recognized that working with "streams" is easier.

Testing

- Finally, you should go through the process of uploading and testing your code on the School of Computing servers.
- Upload your code, any Makefiles, and any test cases to the School of Computing servers. Then login to the School of Computing servers and build and test the code you uploaded to ensure it performs the same as on your machine.
- If your code doesn't compile, or doesn't work the same on the School of Computing servers *you need to figure out why* since we will grade everything on the School of Computing servers.
 - If you can't figure out why, you might need to do all your development directly on the School of Computing servers.

Evaluation

To test your code, examine the evaluation criteria below and test accordingly.

Your program will be graded according to the following rubric:

Command	Output Matches Shell Command
<code>./newecho my PA0 is working</code>	20
<code>./newecho "My program works with quotes"</code>	20
<code>./newcat <file1></code>	20
<code>./newcat <file2></code>	20
<code>./newecho 'echo and cat' > foo.txt;./newcat foo.txt</code>	20
Total = 100 points	

Note: `<file1>` and `<file2>` are simply random files that have **text** in them that we will use.

Evaluation (cont.)

- The output expected in each command is the same output as the built-in `echo` and `cat` shell commands.
- So when in doubt, check what they do to see what your program should do.
- `<file1>` and `<file2>` are simply random files that have **text** in them that we will use.
 - So to test your code generate a few different files with only text in them and make sure the output of your program matches that of the School of Computing server `cat` command.

References (Professional and Ethics)

- Please list all the references in your code.
- If you use a specific function or a section of codes from online resources, please put the link there.
- If you discussed or used someone else's code, please claim their contributions.

Demo

- Makefile
- Commands